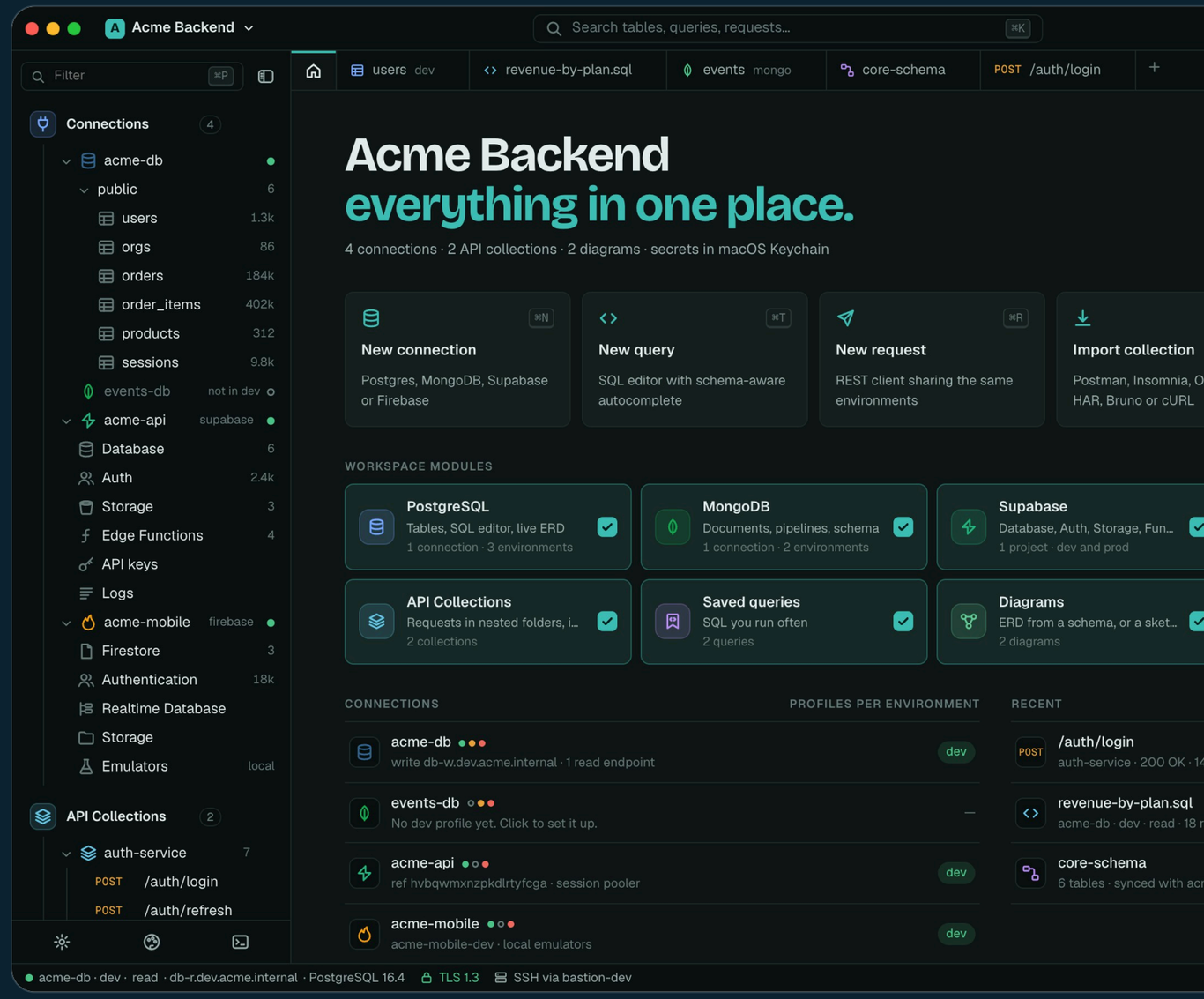
UI/UX CASE STUDY

One harbour for databases and APIs.

How Berth Studio brings a database client and an API client into one calm, fast window that is safe to use on production.



Berth Studio · October 2026

THE PROBLEM

One job, three apps.

Database client

Heavy and crowded. Every environment is a separate connection.

API client

Cloud-first, needs an account, and knows nothing about your database.

Notes

Connection strings for dev, staging and prod, with passwords pasted in too.

The result: constant window switching, the wrong environment, and a little fear every time prod is open.

THE USER

Backend engineers with many environments.

> “At my company we have dev read, dev write, staging read and staging write. All different.”

Feedback from a prototype review, which changed how we model connections

01 Switch environments without opening the wrong database

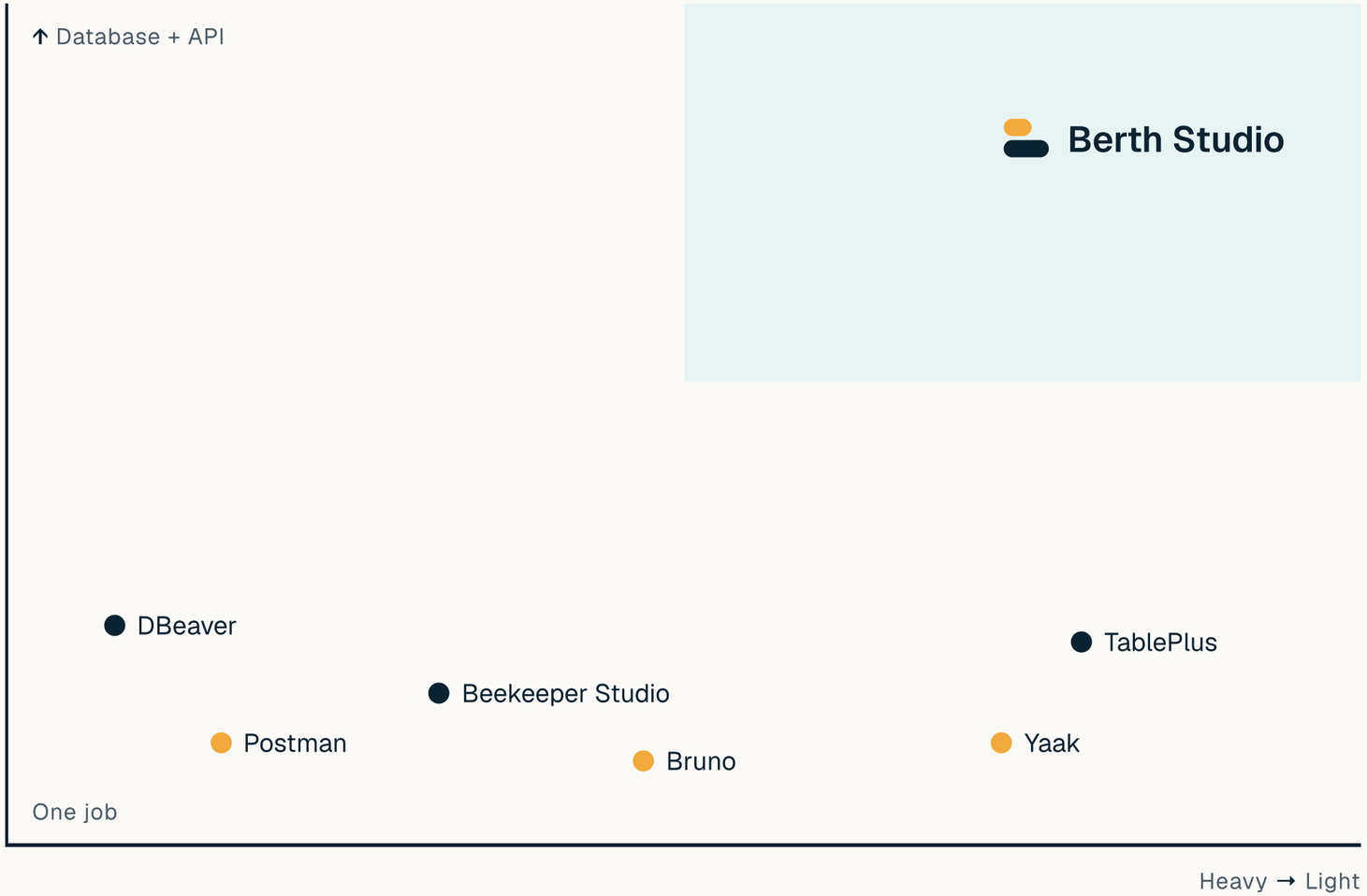
02 Send reads to a replica without thinking about it

03 Be stopped before changing data in prod

04 Use someone else's collection safely

THE LANDSCAPE

The light tools do one thing. The complete ones feel heavy.



- Database client
- API client

Positions are illustrative, based on each tool's technology (JVM, Electron, native, Tauri) and scope.

The gap we fill: as light as a native app, complete for databases and APIs, and aware of environments.

PRINCIPLES

Five rules that settle every debate.

Light

Opens in a second, stays smooth at 100,000 rows.

Local

No account and no cloud. A workspace is just files.

Safe

Passwords live in the system keychain, never on screen.

Keyboard

⌘K for everything.

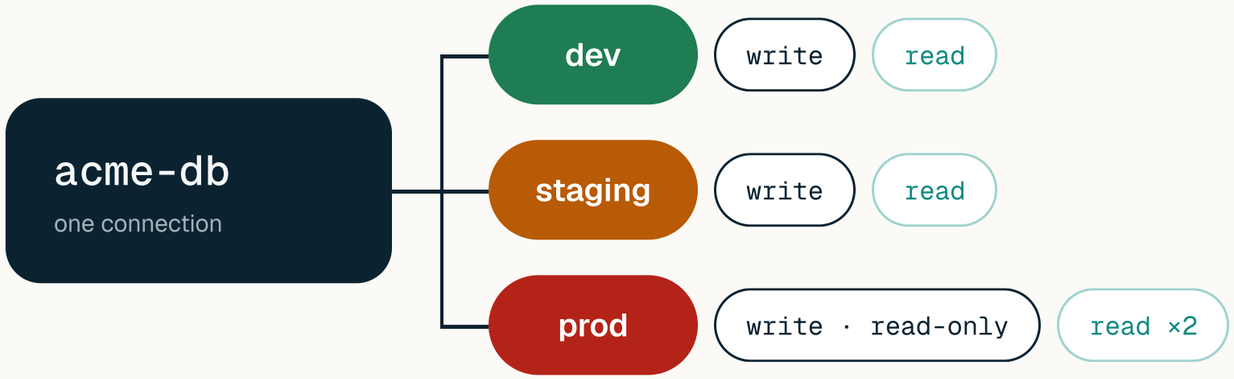
One model

Databases, APIs and diagrams share environments.

Red means production, and only production. If you see red, you are touching prod.

THE CORE IDEA

A connection is a system, not an environment.

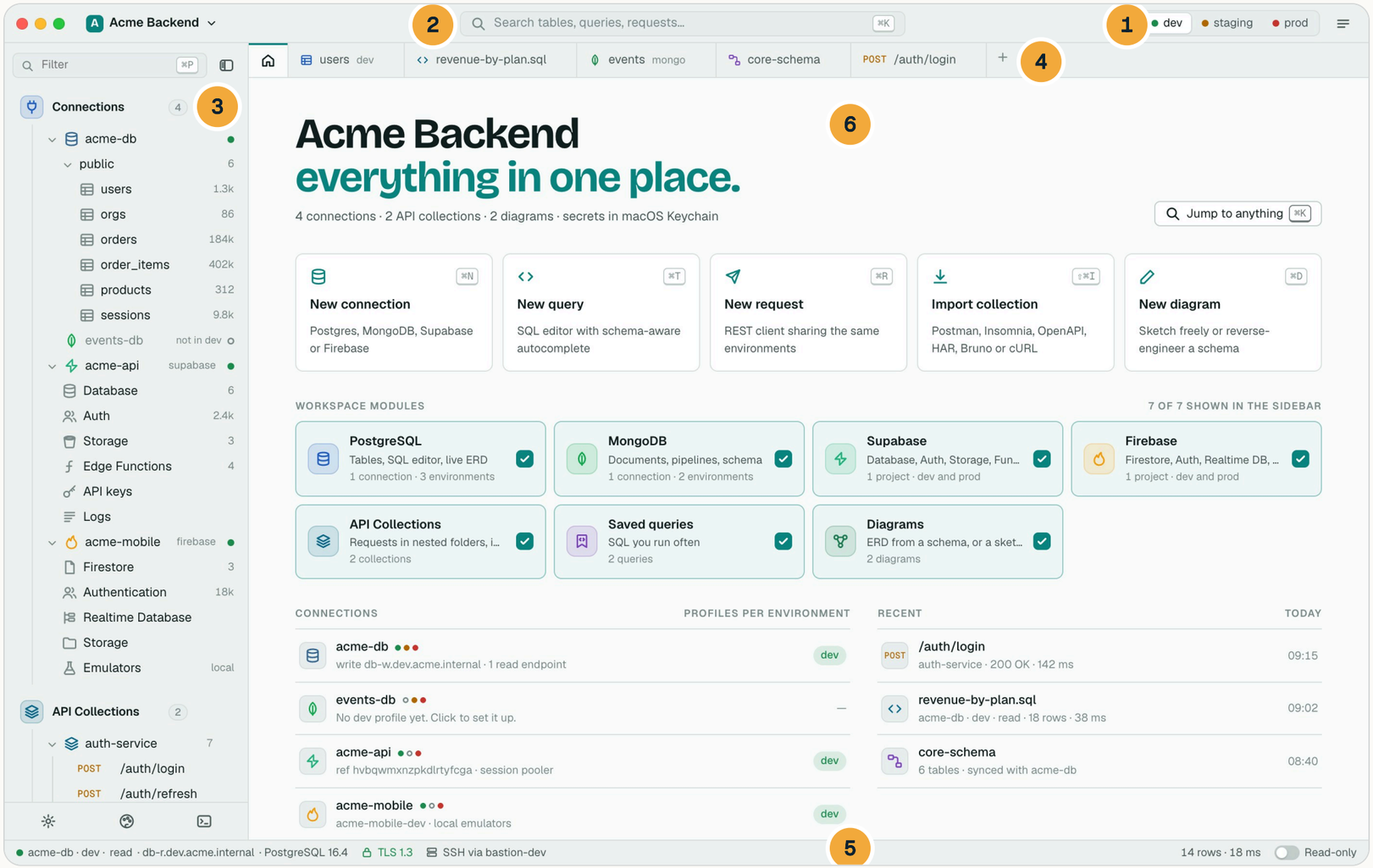


Each connection has one profile per environment. A Postgres profile has one **write** endpoint and any number of **read** endpoints.

The switcher in the title bar changes the profile of every connection at once. When a profile is missing, the app says so plainly.

THE SHELL

One window for every module.



1. **Environment**
   dev, staging, prod for every connection
2. **⌘K**
   find tables, queries, requests, actions
3. **Sidebar**
   connections, API Collections, diagrams
4. **Tabs**
   all your work side by side
5. **Status bar**
   always names the endpoint in use
6. **Welcome**
   quick actions and workspace modules

ENVIRONMENT

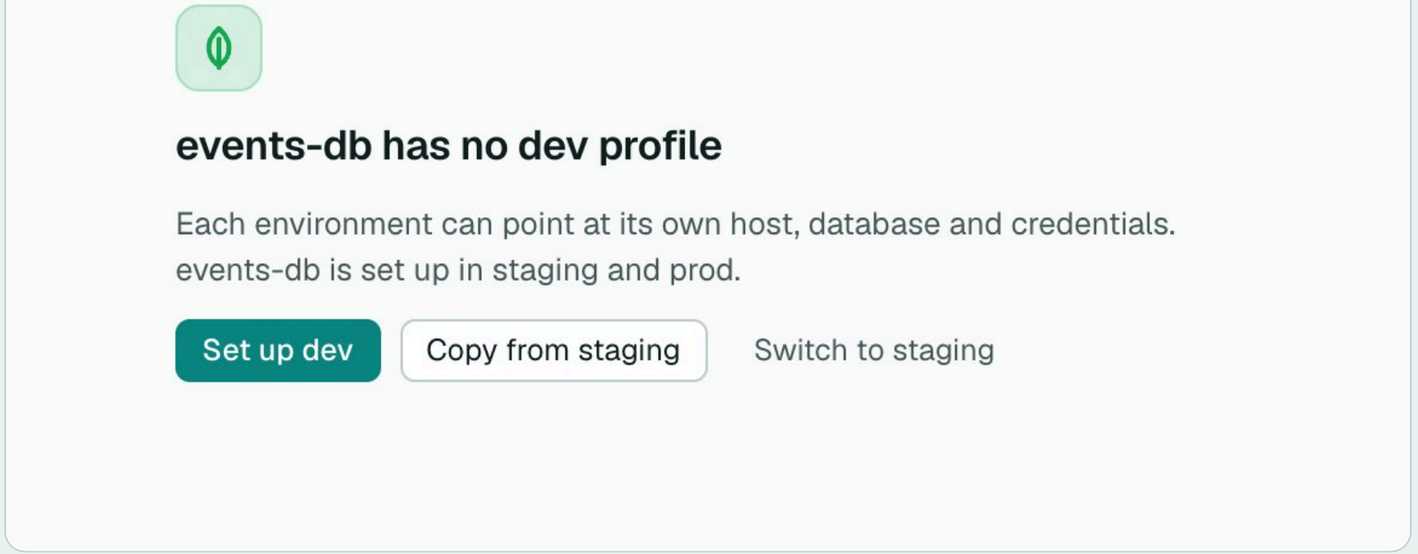
A missing profile is shown honestly.

In dev, events-db has no profile. Instead of guessing, the screen offers three ways forward.

01 **Set up dev**
fill in a new profile

02 **Copy from staging**
hosts and database names are adjusted

03 **Switch to staging**
move to an environment that exists



ENVIRONMENT

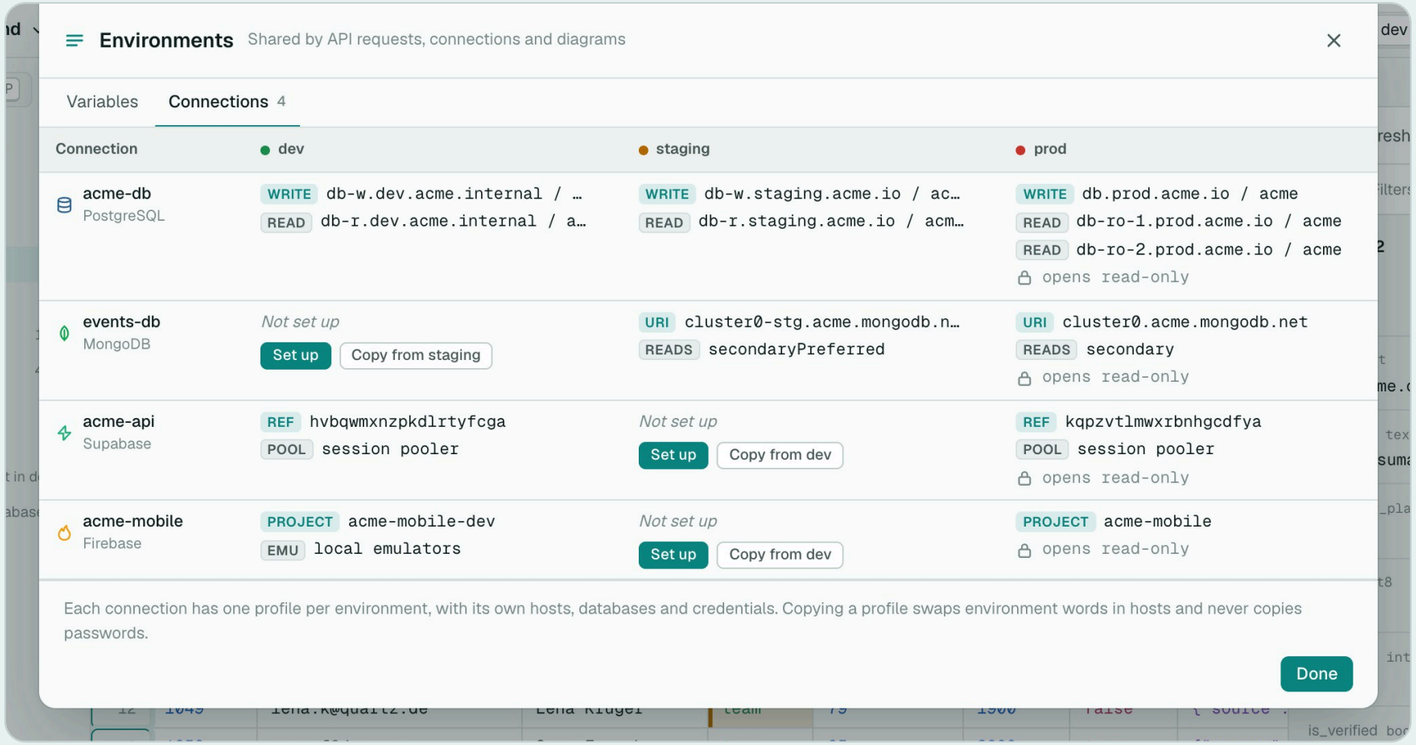
Every profile in one matrix.

Rows are connections, columns are environments. An empty cell can be filled in or copied right there.

01 At a glance
write and read endpoints in every environment

02 Copy from
dev, staging and prod are swapped automatically

03 Safe
passwords are never copied



READ & WRITE

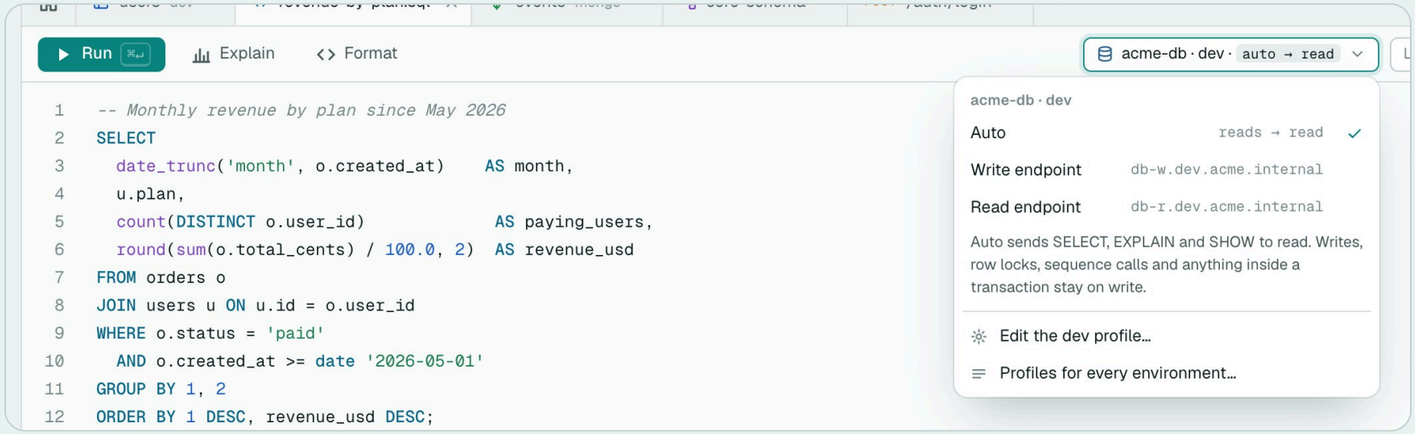
Reads go to a replica. Everything else goes to the primary.

The target chip always says where a query will run. The result names the endpoint that was actually used.

01 **SELECT, SHOW, EXPLAIN**
go to the read replica

02 **Writes, row locks, transactions**
stay on the primary

03 **When in doubt**
the primary, because a replica can lag



READ & WRITE

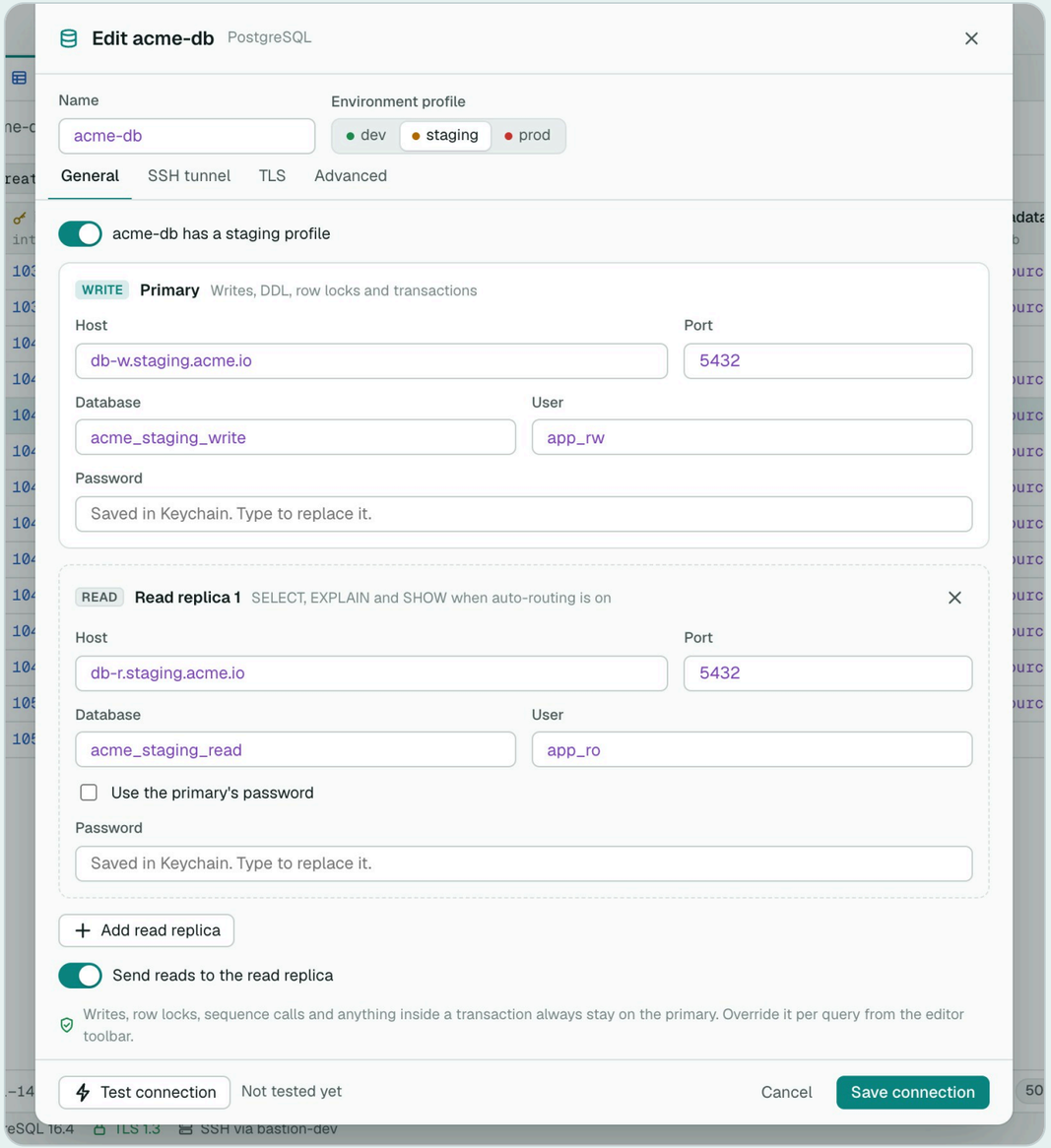
One form, one tab per environment.

A primary is required. Add as many read replicas as you need, each with its own host, database and user.

01 **Profile tabs**
dev, staging and prod in one form

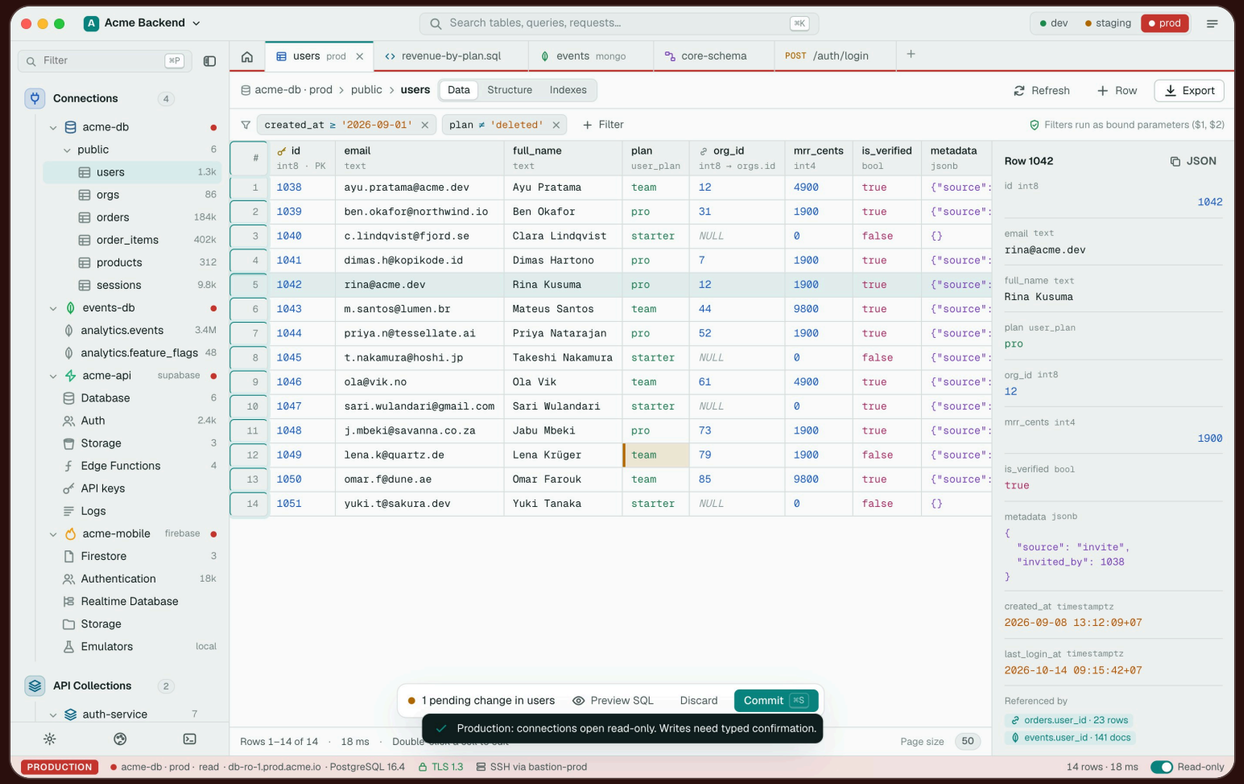
02 **Checked first**
mistakes show up before the connection test

03 **Prod**
always read-only, and it cannot be turned off



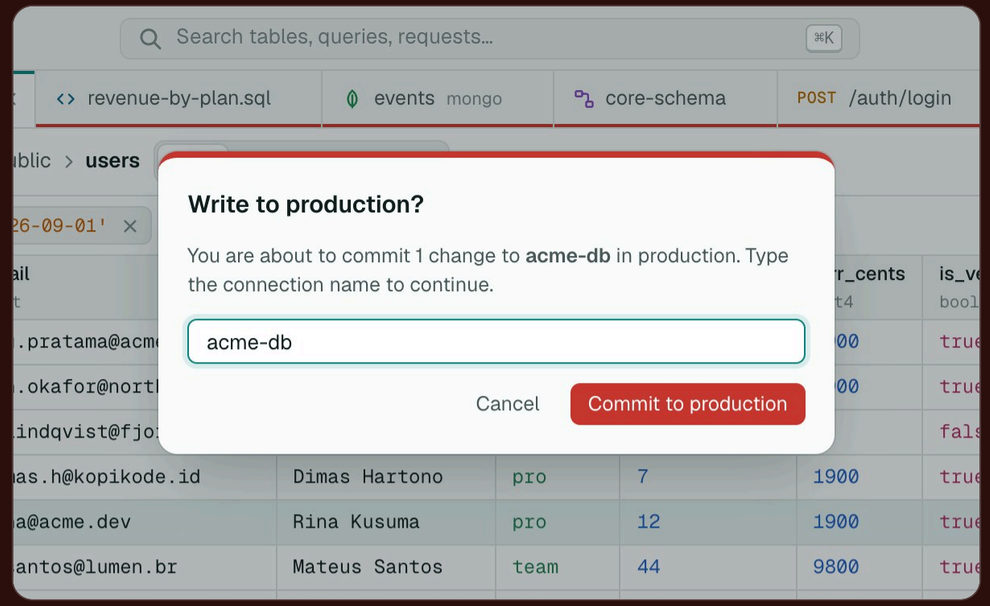
PRODUCTION

Prod has to feel different.



Red and read-only

The tab bar and status bar turn red, and the connection opens read-only.



Type its name first

Committing to prod asks you to type the connection name. The SQL can be reviewed, and everything runs in one transaction.

API COLLECTIONS

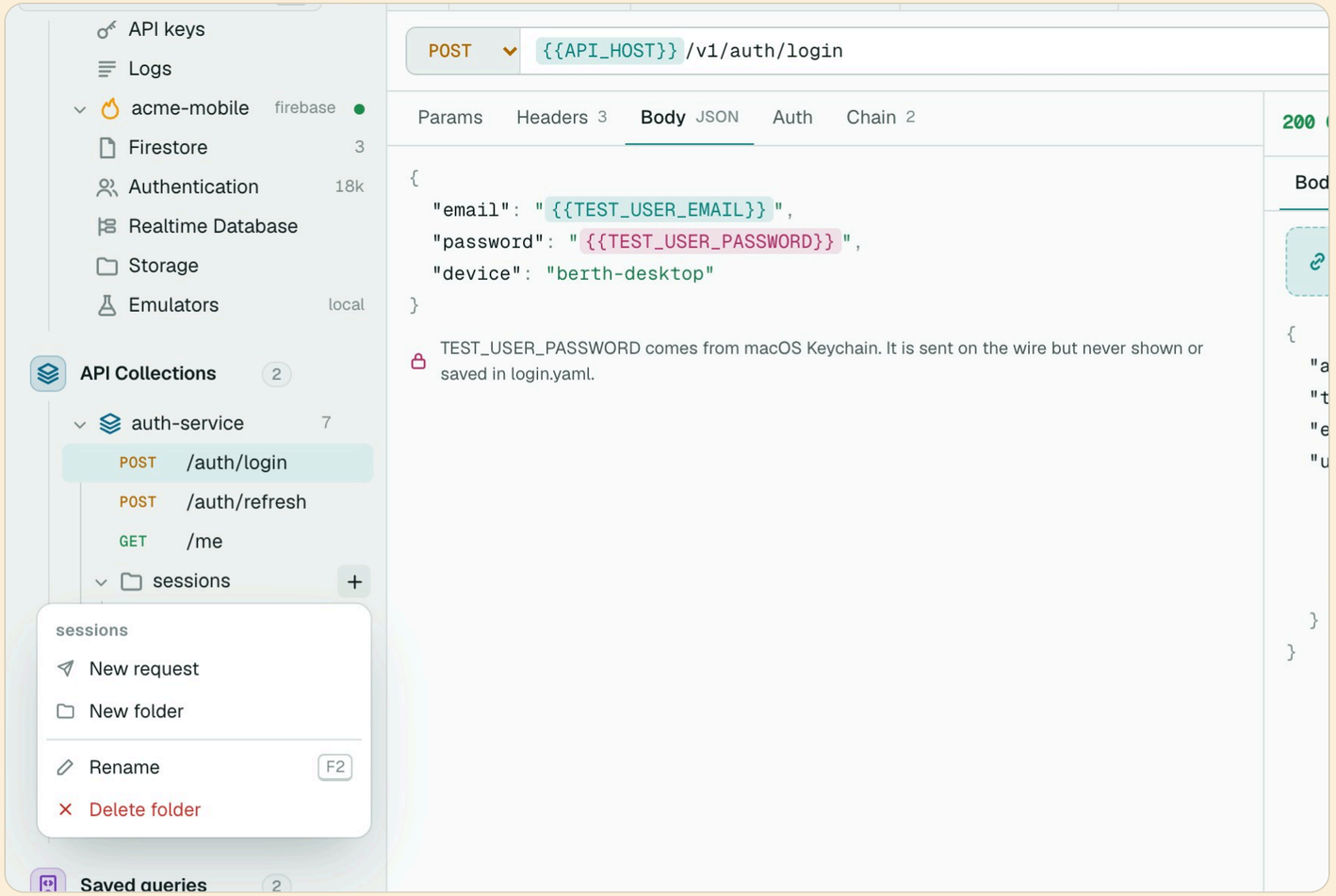
Folders as deep as you like, tidy as a code editor.

Requests sit in nested folders with indent guides, and fold open and shut like in VS Code.

01 **The + button**
New request or New folder right where you are

02 **Undo**
a deleted folder can be brought back

03 **Keyboard**
← → to fold, F2 to rename



IMPORT

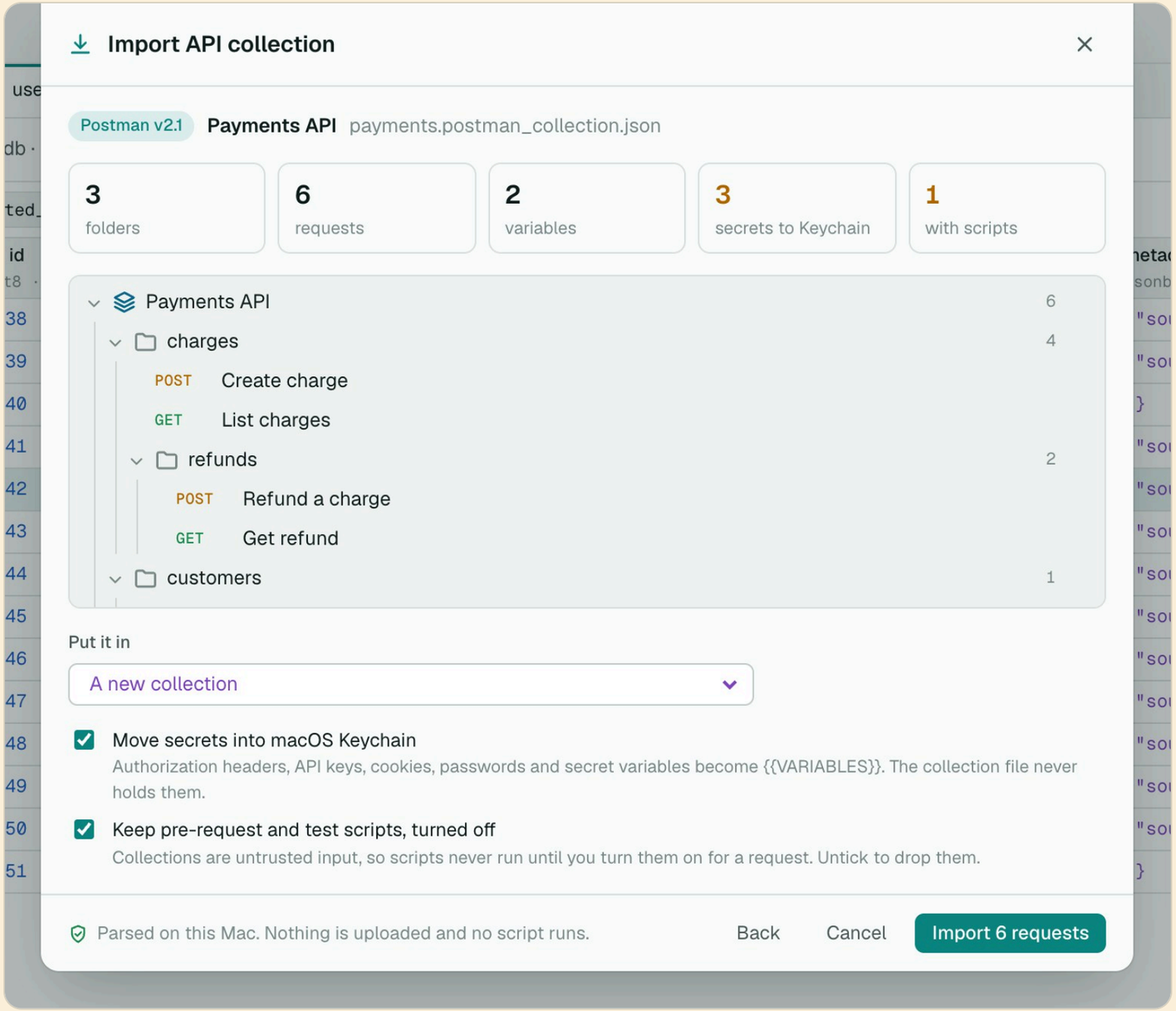
Someone else's collection is treated as untrusted.

Postman, Insomnia, OpenAPI, Swagger, HAR, Bruno and cURL are detected automatically, then checked before anything is saved.

01 **Secrets moved**
tokens become variables, stored in the keychain

02 **Scripts off**
nothing runs until you turn it on

03 **Preview first**
see the tree before it joins your workspace



MODULES

One pattern for every data source.

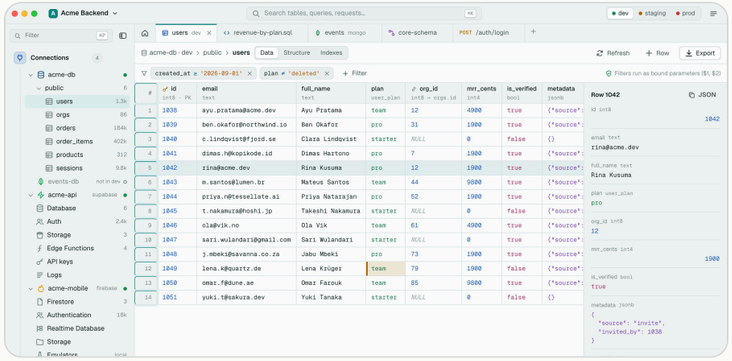
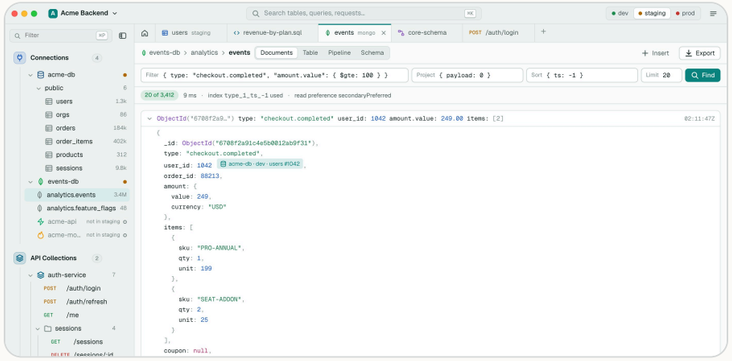
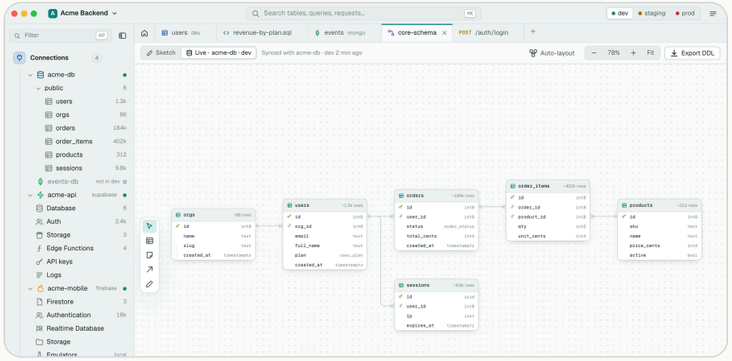


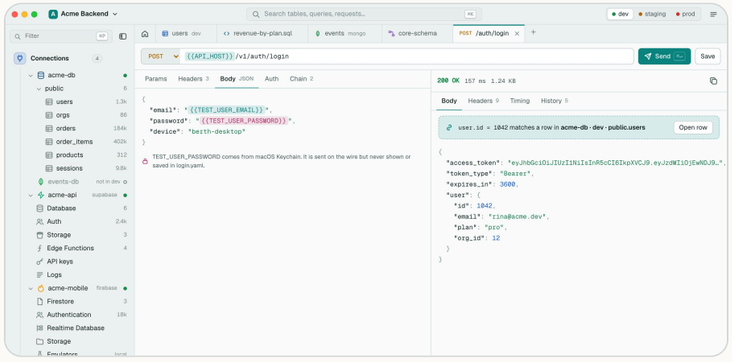
Table browser



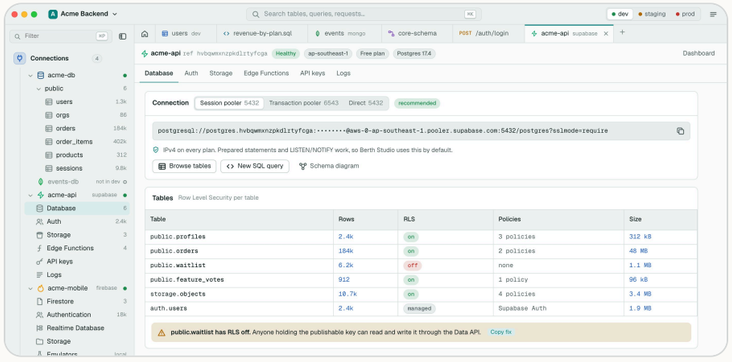
MongoDB



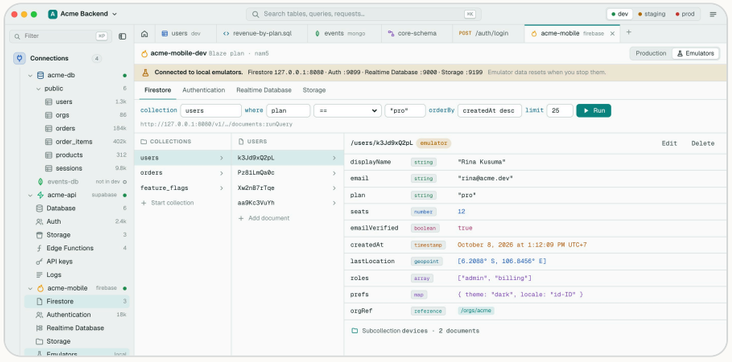
ERD



API client



Supabase



Firebase

VISUAL LANGUAGE

Colour carries meaning, not decoration.

EVERY DATA TYPE HAS ITS OWN COLOUR

`1,520` number	`"pro"` string	`NULL` null	`{ }` json
`2026-10-01` date	`true` boolean	`id` primary key	

The same colours appear in the grid, in JSON and in the SQL editor, so you can read a type without checking the header.

Aa

Geist

For the interface

0123

Geist Mono

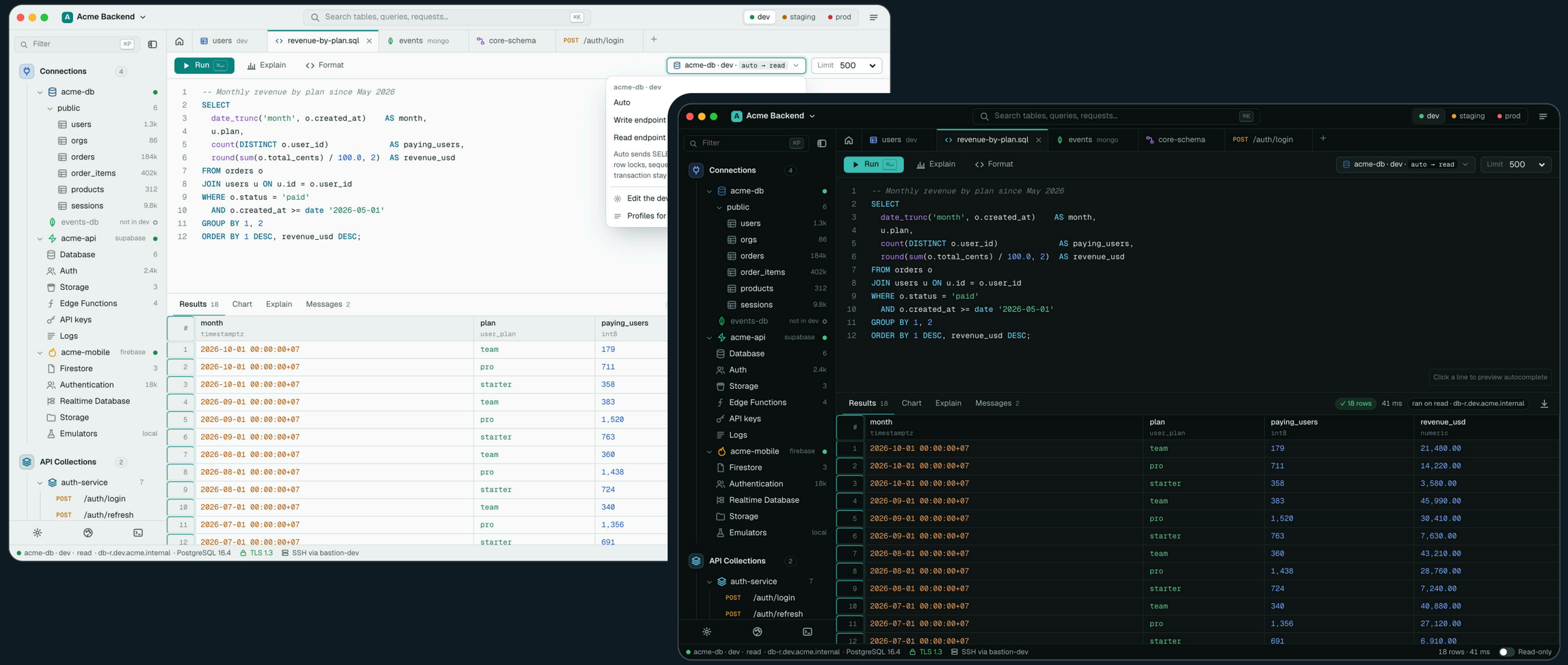
For data and code

One teal accent

For focus and primary actions. Red is only for prod.

THEMES

Light and dark, both first class.

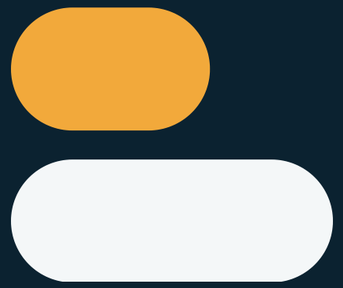


KEYBOARD

Usable without touching the mouse.

⌘K	Find anything
⌘↵	Run a query or Send
⌘.	Cancel
⇧⌘I	Import collection
F2	Rename
Esc	Close

Trees and menus use the right accessibility roles, focus is always visible, and motion stops when reduced motion is on.



One window.
Three environments.
Zero passwords in files.

4

data sources: PostgreSQL, MongoDB, Supabase, Firebase

2

endpoints per environment: read and write, routed automatically

1

colour rule: red means production